Creaera UI · Kickstart

1 / 8 · Loslegen und orientieren

Ein praktischer Einstieg zum Arbeiten mit der Komponentenbibliothek. Stand: 25. Oktober 2026 · Angular 22.2.3 · Nx 24.0.1. Alle Befehle im Repository-Root ausführen. Node.js 24 ab 24.15 und npm werden benötigt.

```
npm ci
npm run storybook
```

Storybook läuft auf Port **6006**. Es braucht dafür weder einen Backend-Dienst noch Zugangsdaten. Starte unter **Foundations → Overview**, probiere dann unter **Components** die einzelne Komponente aus. **Anwenderdoku** erklärt die Nutzung; **Architektur** zeigt den Aufbau einer konsumierenden Anwendung.

Wo liegt was?

Pfad	Zweck
`libs/shared/ui/src/lib`	Wiederverwendbare Komponenten und Direktiven
`libs/shared/ui/src/index.ts`	Öffentliche Exporte für `@creaera/ui`
`libs/shared/ui/src/theme.css`	Zentrale Farben, Abstände und Größen
`libs/shared/ui/src/styles.css`	Globale Komponentenstyles und Tailwind-Import
`libs/storybook/*.stories.ts`	Interaktive Komponentenbeispiele
`libs/storybook/docs`	Anwender- und Architektur-Dokumentation
`libs/storybook/examples/architecture`	Kompilierte Architekturbeispiele

Der erste Arbeitsablauf

1. Die passende Story ansehen und Zustände ausprobieren.
2. Nur die benötigten Komponenten aus `@creaera/ui` importieren.
3. Die globalen Styles einmal in der Anwendung einbinden (Seite 2).
4. Fachliche Daten und Aktionen in der Anwendung halten.
5. Light/Dark Mode, schmale Breite und Tastaturbedienung prüfen.

Prinzip: Kleine Komponenten-APIs; Farben und Größen zentral einstellen. Ein Button braucht keinen eigenen Store und keine HTTP-Verbindung.

Dieses PDF ist eine kompakte Momentaufnahme. Das ausführliche Handbuch `docs/creaera-ui-handbuch.pdf` enthält die vollständigen Erklärungen.

Die erste Komponente verwenden

2 / 8 · Imports und globale Styles

Alle UI-Exporte sind standalone. In jeder verwendenden Angular-Komponente gehören sie in `imports`. Das Alias `@creaera/ui` ist im Workspace eingerichtet.

```
import { Component } from '@angular/core';
import { Button, Card, Checkbox } from '@creaera/ui';

@Component({
  selector: 'app-start',
  imports: [Button, Card, Checkbox],
  template: `
    <ui-card>
      <h2>Benachrichtigungen</h2>
      <ui-checkbox [(checked)]="enabled">Aktivieren</ui-checkbox>
      <button uiButton type="button" (click)="save()">Speichern</button>
    </ui-card>
  `,
})
export class Start {
  enabled = false;
  save(): void {
    console.log(this.enabled);
  }
}
```

Styles einmal global laden

Im Build-Target der Anwendung, beispielsweise in `project.json`:

```
{
  "styles": ["libs/shared/ui/src/styles.css", "apps/my-app/src/styles.css"]
}
```

`apps/my-app` ist ein Beispielpfad; eine solche App ist nicht vorausgesetzt. Die eigene CSS-Datei kommt danach und kann Tokens überschreiben. Die bestehende PostCSS-Konfiguration verwendet `@tailwindcss/postcss`; Storybook lädt die UI-Styles bereits.

Anderes Repository: Das gebaute Paket exportiert `@creaera/ui/styles.css` und `@creaera/ui/theme.css`. Die Styles sind Tailwind-Quell-CSS und benötigen eine passende Tailwind-/PostCSS-Verarbeitung. Alternativ fertiges CSS erzeugen:

```
npx @tailwindcss/cli -i libs/shared/ui/src/styles.css -o dist/ui.css
```

Das ist ein lokaler Build, keine Veröffentlichung in einer Paket-Registry.

Farben, Themes und Größen

3 / 8 · Design an einer Stelle ändern

`libs/shared/ui/src/theme.css` ist die zentrale Konfiguration. Light Mode ist der Standard; primäre Aktionen sind grün, sekundäre grau. Ändere Tokens statt jede Komponente einzeln zu überschreiben.

Tokens	Bedeutung
`--ui-background-primary/secondary/tertiary`	Flächen
`--ui-font-active/default/disabled`	Textfarben
`--ui-interaction-primary/secondary`	Aktionsfarben
`--ui-interaction-hover`, `--ui-interaction-hover-secondary`	Hover
`--ui-interaction-pressed-primary/pressed-secondary`	Gedrückte Aktionen
`--ui-interaction-disabled/divider`	Gesperrte Aktionen und Trennlinien
`--ui-status-normal/caution/dangerous`	Status und Fehler

Die Schrägstriche in der Tabelle kürzen gemeinsame Token-Präfixe ab. Ein eigenes Theme kann in der globalen App-CSS nach den Library-Styles stehen:

```
:root,
.ui-light {
  --ui-interaction-primary: #146c43;
  --ui-interaction-hover: #105b38;
  --ui-interaction-pressed-primary: #0b472b;
}
.ui-dark {
  --ui-interaction-primary: #237a4f;
  --ui-interaction-hover: #1b6942;
  --ui-interaction-pressed-primary: #145534;
}
```

Theme und Größenprofil sind unabhängig

```
<body class="ui-dark ui-large"></body>
```

- Genau eine Theme-Klasse: `ui-light` oder `ui-dark`.
- Ohne Größenklasse: automatisch klein, ab **48rem** groß.
- `ui-small` oder `ui-large` erzwingt ein Profil unabhängig von der Breite.
- Beim Umschalten alte Klassen entfernen, statt beide gleichzeitig zu setzen.

Die Größenprofile definieren unter anderem `--ui-padding`, `--ui-margin`, `--ui-button-height`, `--ui-button-padding` und `--ui-font-size`. Die Buttonhöhe beträgt aktuell 44px bzw. 48px bei üblicher Root-Schriftgröße. Für Änderungen beide Profile berücksichtigen; später definierte Profilregeln können allgemeine `:root`-Werte überschreiben.

Cards bleiben auch im Dark Mode weiß: `--ui-card-background` und `--ui-card-text` steuern ihre Fläche und Textfarbe. Nach jeder Farbänderung Textkontrast sowie Hover, Fokus und Fehlerzustände prüfen.

Buttons, Auswahl und Menüs

4 / 8 · Die passende Interaktion wählen

Die folgenden Ausschnitte gehören in das Template einer Komponente. Die jeweiligen Exporte aus `@creaera/ui` zusätzlich in deren `imports` aufnehmen.

```
<button uiButton type="button">Speichern</button>
<button uiButton variant="secondary" type="button">Abbrechen</button>
<button uiButton variant="tertiary" type="button">Details</button>
<button uiButton variant="text" type="button">Mehr erfahren</button>
<button uiButton type="button" disabled>Gesperrt</button>
```

Icons als Inhalt projizieren, etwa als SVG mit `aria-hidden="true"`. Ein reiner Icon-Button braucht einen zugänglichen Namen über `aria-label`. Innerhalb eines Formulars `type="button"` für Nebenaktionen und `type="submit"` für die eigentliche Absendeaktion verwenden.

Boolesche Werte oder eine feste Auswahl

```
<ui-checkbox [(checked)]="accepted">Bedingungen akzeptieren</ui-checkbox>
<ui-toggle-switch [(checked)]="notifications">Benachrichtigungen</ui-toggle-switch>
```

`Checkbox` und `ToggleSwitch` unterstützen auch Angular Forms. `Radio` wird auf einem nativen `input type="radio"` `uiRadio` verwendet; Radios einer Gruppe bekommen denselben `name` und unterschiedliche Werte.

```
options = [
  { value: 'list', label: 'Liste' },
  { value: 'grid', label: 'Kacheln' },
];
selected: readonly string[] = ['list'];
```

```
<ui-button-group label="Ansicht" [options]="options" [(selected)]="selected" />
```

`ButtonGroup` verwendet **auch bei Einfachauswahl ein ID-Array**. Für Mehrfachauswahl `[multiple]="true"` ergänzen.

Aktionen, keine Formular-Selects

```
<ui-dropdown label="Aktionen" [options]="actions" (selected)="runAction($event)" />
<ui-option-button
  label="Speichern"
  [options]="actions"
  (action)="save()"
  (selected)="runAction($event)"
/>
```

`actions` ist eine Liste von `UiOption`: `{ value, label, disabled? }`. `selected` liefert den String-Wert. Der `OptionButton` trennt Hauptaktion und Menüpfeil. Beide Menüs sind Aktionsmenüs, keine FormControls für Wertauswahl.

Formulare und Autocomplete

5 / 8 · Native Felder, klare Fehler

Ein `FormField` umschließt genau ein direktes natives `input uiInput` oder `textarea uiInput`. `label` ist erforderlich. Angular Forms bindet am nativen Feld; Attribute wie `type`, `autocomplete`, `min`, `readonly` bleiben erhalten.

```
import { Component } from '@angular/core';
import { FormControl, FormGroup, ReactiveFormsModule, Validators } from '@angular/forms';
import { Button, FormField, Input, Autocomplete } from '@creaera/ui';

@Component({
  selector: 'app-contact',
  imports: [ReactiveFormsModule, Button, FormField, Input, Autocomplete],
  template: `
    <form [formGroup]="form" (ngSubmit)="submit()" novalidate>
      <ui-form-field label="E-Mail">
        <input uiInput type="email" autocomplete="email" [formControl]="email" />
      </ui-form-field>
      <ui-autocomplete label="Stadt" [options]="cities" [formControl]="city" />
      <button uiButton type="submit">Übernehmen</button>
    </form>
  `,
})
export class Contact {
  readonly email = new FormControl('', {
    nonNullable: true,
    validators: [Validators.required, Validators.email],
  });
  readonly city = new FormControl('', { nonNullable: true });
  readonly form = new FormGroup({ email: this.email, city: this.city });
  readonly cities = ['Berlin', 'Bern', 'Hamburg'];
  submit(): void {
    this.email.markAsTouched();
    if (this.email.invalid) return;
    console.log({ email: this.email.value, city: this.city.value });
  }
}
```

Ungültige Controls erhalten nach `dirty` oder `touched` eine rote Border und eine allgemeine Fehlermeldung. Ein eigener Text über `[error]="message"` aktiviert den Fehlerzustand sofort; `''` entfernt den expliziten Fehler. Für größere Formulare `FormGroup` und `markAllAsTouched()` verwenden.

Autocomplete: `options` ist ein `readonly string[]`; lokale Filterung, Tastaturbedienung und freie Texteingabe sind enthalten. Alternativ zu Forms ist `[(value)]` möglich. Nicht beide Bindungswege gleichzeitig verwenden. Server-Ergebnisse können als Optionen geliefert werden. Eine eigene Query-/ Loading-API und ein Abschalten der lokalen Filterung sind noch nicht umgesetzt. Die Anwendung verantwortet Requests, Stornierung und fachliche Validierung.

Cards, Chips und hierarchische Daten

`Card`, `Chip`, `Tree` und `TreeTable` aus `@creaera/ui` importieren. Bei einer ausklappbaren Card bleibt der mit `uiCardHeader` markierte Bereich sichtbar; der restliche Inhalt klappt auf und zu.

```
<ui-card collapsible label="Projekt" border="secondary">
  <h2 uiCardHeader>Projektübersicht</h2>
  <p>Details zum Projekt.</p>
  <ui-card><p>Verschachtelte Card</p></ui-card>
</ui-card>
```

Ausklappbare Cards zeigen den Stapel aus drei Ebenen und den Pfeil unten mittig. Sie sind initial geschlossen; `[expanded]="true"` oder `[(expanded)]="open"` steuert den Zustand. `border` ist `primary` oder `secondary`.

```
@for (tag of tags; track tag) {
<ui-chip (removed)="removeTag(tag)">{{ tag }}</ui-chip>
}
```

Der kleine X-Button meldet `removed`. **Die Anwendung entfernt den Eintrag** aus `tags`; der Chip löscht seine Daten nicht selbst. `disabled` sperrt das Entfernen.

Ein Datenmodell für Tree und Tree Table

```
import type { UiTreeNode, UiTreeColumn } from '@creaera/ui';

nodes: UiTreeNode[] = [{
  id: 'project', label: 'Projekt',
  children: [{
    id: 'buttons', label: 'Buttons', data: { status: 'Fertig' },
  }],
}];
columns: UiTreeColumn[] = [{ key: 'status', label: 'Status' }];
openIds: readonly string[] = ['project'];
selectedId: string | null = null;
```

```
<ui-tree label="Projektstruktur" [nodes]="nodes" [(expanded)]="openIds" [(selected)]="selectedId" />
<ui-tree-table
  label="Projektstatus"
  [nodes]="nodes"
  [columns]="columns"
  [(expanded)]="openIds"
  [(selected)]="selectedId"
/>
```

IDs müssen im gesamten Baum eindeutig sein. Zusatzspalten lesen `data[column.key]`; die erste Spalte zeigt immer den Namen und die Hierarchie. Pfeiltasten navigieren und klappen auf/zu, Enter oder Leertaste wählen aus. Fokus und Auswahl sind getrennt. Ein gesperrter Knoten sperrt seine Nachfahren.

Eigene Komponenten ergänzen

7 / 8 · Vom Generator zur öffentlichen API

Generische UI gehört nach `libs/shared/ui/src/lib`, Fachlogik in die Anwendung. Erst Zweck und minimale Inputs/Outputs festlegen; dann einen Dry-Run ansehen:

```
npx nx g @nx/angular:component libs/shared/ui/src/lib/status-badge/status-badge \
  --selector=ui-status-badge --standalone --changeDetection=OnPush \
  --style=none --inlineTemplate --export --dry-run
```

Für die echte Anlage `--dry-run` entfernen. Nx 24 erwartet hier einen Dateipfad. Die Ausgabe und eventuelle Änderungen an `nx.json` prüfen. Das folgende Badge ist **ein Beispiel für eine neue Komponente**, kein bereits exportiertes UI-Control:

```
import { ChangeDetectionStrategy, Component, input } from '@angular/core';

@Component({
  selector: 'ui-status-badge',
  changeDetection: ChangeDetectionStrategy.OnPush,
  template: '<span>{{ label() }}</span>',
  host: { '[style.color]': "'var(--ui-status-' + tone() + ')'" },
})
export class StatusBadge {
  readonly label = input.required<string>();
  readonly tone = input<'normal' | 'caution' | 'dangerous'>('normal');
}
```

`--export` ergänzt den Library-Einstiegspunkt. Falls nötig manuell in `libs/shared/ui/src/index.ts` exportieren. Keine internen Dateipfade in konsumierenden Anwendungen importieren.

Eine Story macht die neue API sichtbar

Nach dem Anlegen und Exportieren unter `libs/storybook/status-badge.stories.ts`:

```
import type { Meta, StoryObj } from '@storybook/angular';
import { StatusBadge } from '@creaera/ui';
const meta: Meta<StatusBadge> = {
  title: 'Components/Status Badge',
  component: StatusBadge,
  args: { label: 'Bereit', tone: 'normal' },
};
export default meta;
export const Default: StoryObj<StatusBadge> = {};
```

Vor dem PR: Export und Story prüfen, Verhalten sinnvoll testen und alle vorhandenen Projektchecks ausführen. Themes, Größenprofile, Disabled-/Fehler- Zustände und Tastaturbedienung berücksichtigen. Keine eigenen Farb-Inputs anlegen, wenn ein zentraler Token den Bedarf abdeckt.

Arbeitsroutine und Architektur

8 / 8 · Befehle, Zuständigkeiten und schnelle Hilfe

Befehl	Wann verwenden?
`npm run storybook`	Komponenten interaktiv entwickeln
`npm test`	UI-Komponententests ausführen
`npm run build`	Library nach `dist/libs/shared/ui` bauen
`npm run build-storybook`	Statische Website nach `dist/storybook` bauen
`npm run test:e2e`	Storybook bauen und Browser-Tests ausführen
`npm run format:check`	Formatierung prüfen
`npx nx show projects`	Vorhandene Nx-Projekte auflisten
`npx nx show project ui`	Targets der UI-Library ansehen

Für Browser-Tests einmalig `npx playwright install chromium` ausführen. Für Änderungen an Architekturbeispielen zusätzlich:

```
npm run docs:architecture:sync
npm run docs:architecture:check
npm run test:architecture
```

Nx oder ng?

Hier `npx nx ...` verwenden. `g` bedeutet `generate`; `--dry-run` zeigt Änderungen, `--help` die Optionen. Klassische Angular-CLI-Workspaces mit `angular.json` verwenden dagegen etwa `npx ng g c ...`, `npx ng g s ...`, `npx ng build` und `npx ng test`. Die UI-Library wird in Storybook betrachtet, nicht per App-Server.

Eine einfache Anwendung sauber aufteilen

UI → Feature-Page → Feature-Store → API-Client → Server

- UI stellt dar und meldet Absichten über Outputs; sie kennt keine Endpunkte.
- Die Page verbindet UI, Formulare und fachliche Aktionen.
- Ein Feature-Store hält geteilten Zustand; lokale UI-Zustände bleiben lokal.
- NgRx Signal Store bei Bedarf am Feature bereitstellen; `computed` für abgeleitete Werte, fachliche Methoden für Änderungen verwenden.
- API-Clients bündeln HTTP und DTO-Mapping. Der Server prüft Geschäftsregeln.
- Überholte Suchanfragen stornieren; Doppelaktionen beim Schreiben verhindern. Idempotenz und Versionskonflikte gehören auch zum Serververtrag.

Wenn etwas nicht klappt

Unbekanntes Element: Export und `imports` prüfen. **Keine Gestaltung:** globale Styles und Tailwind-Verarbeitung prüfen. **Wert aktualisiert sich nicht:** passende `[(...)]`-Bindung oder Angular Forms verwenden. **Fehler bleibt unsichtbar:** Control berühren/validieren; `[error]` für explizite Meldungen setzen. **Server-Suche gewünscht:** Die Anwendung steuert Requests; Autocomplete bietet aktuell lokale Textfilterung. Details stehen im großen Handbuch und in Storybook.